SMS Weather Station: Project Guide

EEIC/7301 Project Design & Evaluation. Status as of 23 October 2026.

This guide explains, in plain language, what the project does, how it is built, what has been tested so far, and how to run it. No programming knowledge is needed to follow it. The full technical report, with the theory behind each part, circuit explanations and IEEE references, is `report/PROJECT_REPORT.pdf`.

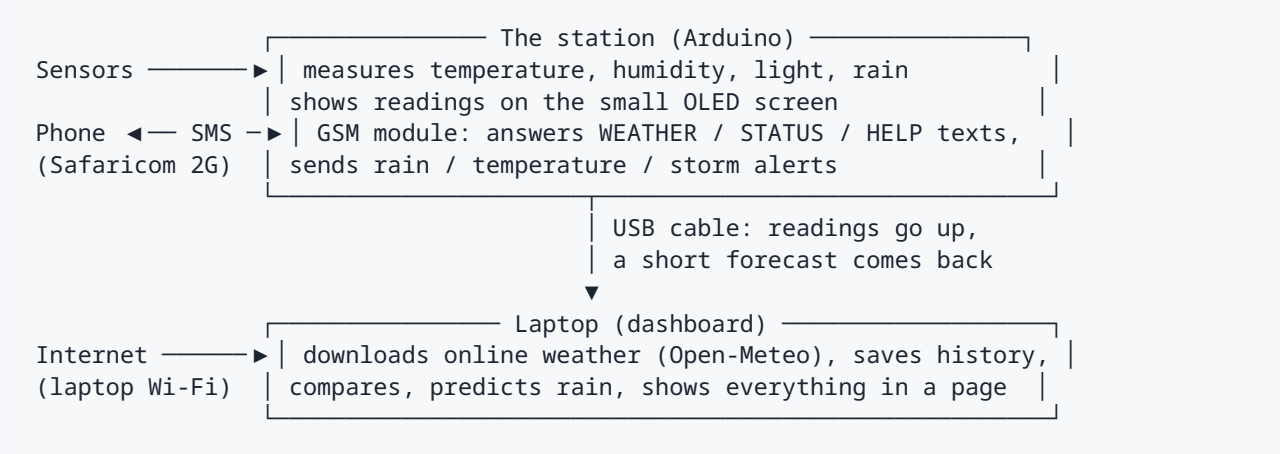
---

1. The idea in one paragraph

We built a small **weather station** from an Arduino and some sensors. It measures **temperature, humidity, light and rain**, shows them on a small screen, and **answers text messages (SMS)**: anyone on an approved list can text the word `WEATHER` to the station's SIM card and get the current conditions back on their phone, including a short online forecast. The station also texts **warnings** by itself, for example when it starts raining. When it's plugged into a laptop, a **dashboard web page** shows live readings, compares them with official online weather data, predicts rain, and measures how accurate the online forecast is for our location.

---

2. How the pieces fit together



Two separate connections, two separate jobs:

- The **GSM module** (SIM800L) uses the **mobile network for SMS only**. It does not connect to the internet.
- The **laptop** uses **its own internet** (Wi-Fi, cable or hotspot) to download online weather.

The SMS service works on its own, even with the laptop switched off. The laptop adds the dashboard and the online forecast in the SMS replies.

---

3. The parts and what each one does

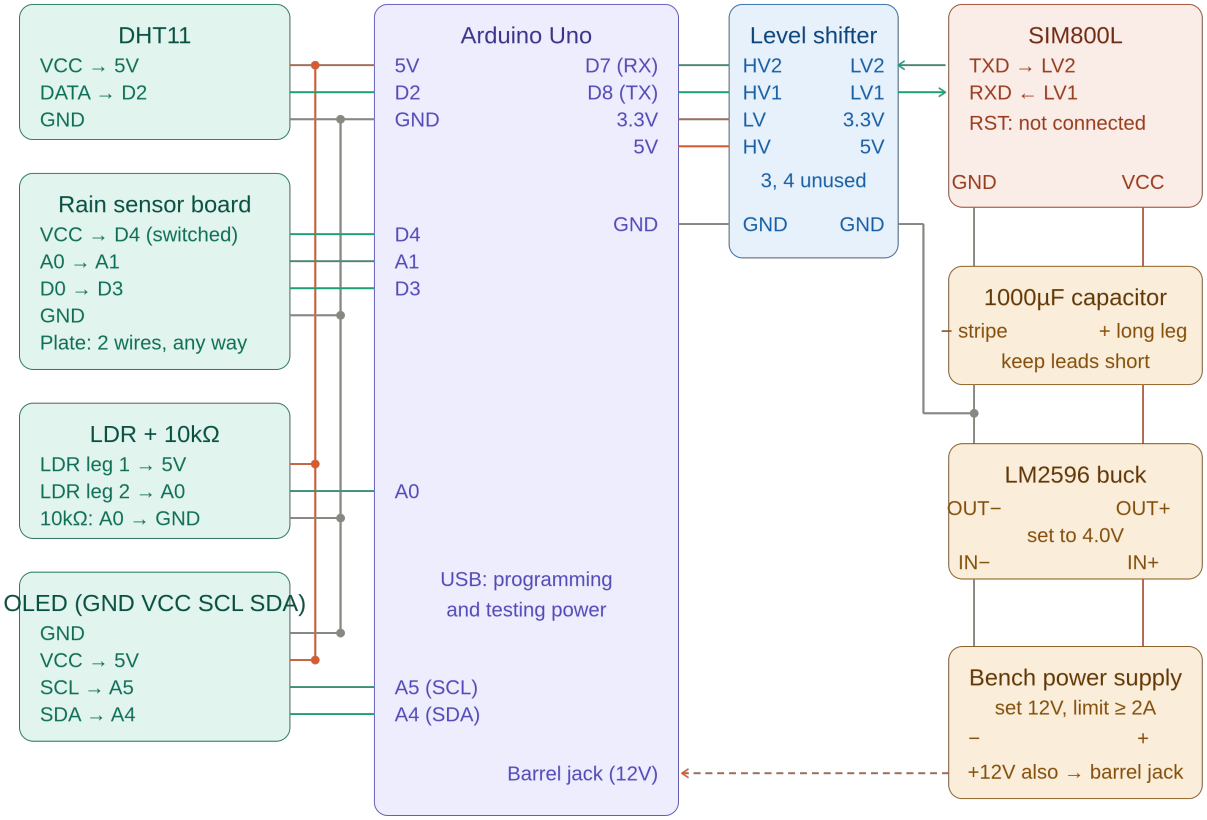
Part	What it does
Arduino Uno	The small computer that runs everything on the station
DHT11	Measures temperature (0–50 °C) and humidity (20–90 %)
LDR (light sensor) + 10 kΩ resistor	Measures how bright it is
Rain sensor (plate + small board)	Detects water on the plate: dry, damp, light rain, heavy rain
OLED screen (0.96", 128×64)	Shows the readings on the station
SIM800L GSM module + antenna + SIM	Sends and receives SMS on the Safaricom network
LM2596 power converter	Turns the 12 V supply into the 4.0 V the GSM module needs
1000 µF capacitor	Smooths the GSM module's power when it transmits
Logic level shifter	Lets the 5 V Arduino talk safely to the 2.8 V GSM module
Bench power supply (12 V)	Powers the GSM module through the LM2596

Why the GSM module needs special power: when it transmits, it briefly draws up to 2 A, far more than the Arduino can supply. Without its own supply it keeps restarting.

---

4. Wiring summary

The diagram shows every connection, including the level shifter between the Arduino and the GSM module. Its notes at the bottom explain the colours: orange = 5V, grey = ground, and dots are joined wires.



Left rails: orange = 5V, grey = GND. Dots are joined wires.
Level shifter: HV side faces the Arduino (5V), LV side faces the SIM800L (3.3V).
Grey line from the shifter GND to OUT− is the common ground. Never skip it.
Dashed line: in the final build the 12V also powers the Uno, so USB is not needed.
Without the shifter: D8 → 1kΩ → SIM RXD, 2.2kΩ from SIM RXD to GND; SIM TXD → D7 direct.
SIM800L must never see 5V on VCC. Check 4.0V with a multimeter before connecting it.

Wiring diagram (`sms_weather_station_wiring_level_shifter.png`). The same connections as a table:

Arduino pin	Connected to
D2	DHT11 data
D3	Rain sensor digital output (D0)
D4	Rain sensor power. Switched on only while measuring, so the plate corrodes more slowly.
D7	GSM module TXD (through the level shifter)

Arduino pin	Connected to
D8	GSM module RXD (through the level shifter)
A0	Light sensor (middle point between the LDR and the 10 kΩ resistor)
A1	Rain sensor analog output (A0)
A4 / A5	OLED screen SDA / SCL
5V	DHT11, OLED, light sensor, level shifter high side
3.3V	Level shifter low side
GND	All grounds, **including the LM2596 output minus**

Important rules:

- Set the LM2596 to **4.0 V with a multimeter before** connecting the GSM module. 5 V can destroy it.
- Never power the GSM module from the Arduino's 5V or 3.3V pins.
- The GSM module's ground must be joined to the Arduino's ground ("common ground"), or they cannot communicate.

Full wiring details are in `sms-weather-station-notes.md`.

---

5. What has been built

The project is in three parts, built and tested in this order.

Part 1: Module tests (`tests/` folder)

Ten small test programs, each checking **one part on its own** before everything is combined. Each one prints a clear PASS or FAIL message.

#	Tests	Result
01	Arduino, USB cable, serial connection	✓ Pass
02	Finds the OLED screen on its connection bus (address 0x3C)	✓ Pass
03	OLED screen draws text and graphics	✓ Pass
04	DHT11 temperature and humidity (28 °C, 46 %)	✓ Pass
05	Light sensor (≈880 of 1023 in room light)	✓ Pass
06	Rain sensor (≈1010–1020 when dry)	✓ Pass
07	GSM module health check	× Module faulty (see section 7)

#	Tests	Result
08	Send an SMS	⏳ Waiting for a working GSM module and SIM
09	Receive an SMS and reply	⏳ Waiting for a working GSM module and SIM
10	All sensors and the screen together	✓ Pass

Part 2: Station program (`firmware/weather_station/`)

This is the program that runs on the Arduino permanently. It:

- **Reads all sensors every 5 seconds.**
- **Shows readings on the OLED screen**: temperature, humidity, light, rain, and how long the station has been running. GSM details (signal, network) are only in the `STATUS` text reply.
- **Sends each reading to the laptop** over the USB cable.
- **Remembers the last hour of readings** (one every 5 minutes) to work out trends such as "humidity rising".
- **Answers SMS commands** from approved numbers, and ignores everyone else:

You text	You get back
`WEATHER`	`Weather 14/10 12:30: Temp 24.5C, Hum 61%, Light 72%, Rain: dry. Trend: humidity rising. Fcst: rain 40% next 3h.`
`STATUS`	Network, signal strength, how long it has been running, free memory, time of the last SMS
`HELP` (or anything else)	The list of commands

- **Sends alerts automatically** to the approved numbers:
  - when **rain starts**
  - when the **temperature goes above 30 °C or below 10 °C**
  - when the **light drops sharply** (possible storm)

  Each kind of alert is sent at most **once every 30 minutes**, so phones aren't flooded.
- **Can send an hourly report** to subscribers (switched off by default).
- **Keeps working if the GSM module fails.** It retries the module every 30 seconds and carries on measuring meanwhile.

The "Fcst" part of the `WEATHER` reply comes from the laptop (section 2). If the laptop has been disconnected for more than 90 minutes, the reply is sent without it.

All settings are in one file, `config.h`: approved phone numbers, alert limits, sensor calibration and timings.

Memory note: the Arduino Uno has only 2 KB of working memory. The original screen library used half of it, so the station program uses a lighter, text-only way of driving the screen. About 760 bytes stay free while running, which is a safe margin.

Part 3: Dashboard (`dashboard/` folder)

A web page that runs on the laptop and opens in the browser at **http://localhost:5000**. On a wide screen the first three sections sit side by side, then two, then the chart across the full width, then two more. On a phone they stack in one column.

Section	What it shows
Now at this location	Latest station readings: temperature, humidity, light and rain in a 2×2 grid
Online conditions and forecast	Current weather and the next 12 hours from Open-Meteo (a free weather service)
Comparison	Station vs online temperature and humidity, and the difference
Rain prediction	*Unlikely*, *possible* or *likely*, with the reasons listed
SMS phone	Whether the Android backup phone (section 7) is connected (green) or not (red), plus its address, login and approved numbers; change and save them here
Last 24 hours	Chart of temperature, humidity and light. The temperature scale always covers at least 6 °C, so the DHT11's normal flicker of a few tenths of a degree doesn't look like big jumps
Forecast accuracy	How close the online forecast was to what the station actually measured
SMS log	Who texted what, and the replies and alerts sent

Everything is saved in a small database file (`dashboard/data.db`), so history survives restarts.

How the rain prediction works. It adds up points for warning signs:

- high humidity, or humidity rising over the last hour
- light dropping sharply (clouds building)
- the rain sensor being damp or wet
- the online forecast's chance of rain in the next 3 hours

A low total means *unlikely*, a medium total *possible*, a high total *likely*. A pressure sensor (BMP280) can be added later as one more warning sign; the dashboard is already prepared for it.

How forecast accuracy is measured. For every finished hour, the dashboard compares:

- the station's **average** temperature and humidity for that hour, with
- what Open-Meteo **predicted for that hour at least one hour earlier**.

It reports the **average error** ("MAE": for example, "the forecast was off by 1.8 °C on average") for the last 24 hours and the last 7 days. It also reports **how often the forecast was right about rain**. These numbers are useful evaluation results for the project report. They start empty and fill in over a few hours of running.

---

6. How to run it

The dashboard (one command)

Computer	What to do
Linux / macOS	Open a terminal in the project folder and run `./run.sh`
Windows	Double-click **`run.bat`**. It installs Python by itself if it's missing

It sets itself up the first time (internet needed for that), starts the dashboard, and opens it in the browser. The step-by-step Windows guide, including the USB driver some Arduino clones need, is in `README.md` ("Set up on Windows").

- **Without the station:** add `--fake` (`./run.sh --fake` or `run.bat --fake`) to show realistic made-up readings.
- **To stop it:** close the dashboard's browser tab. Everything it started stops about 10 seconds later; reloading the page doesn't count as closing. Ctrl+C in the terminal also works. Exception: while the phone SMS backup (section 7) is switched on, closing the tab doesn't stop it, so texts keep being answered. Use Ctrl+C then.

Before starting, close the Arduino IDE's Serial Monitor. Only one program can use the station's USB connection at a time.

The station

The station program is already loaded on the Arduino and starts by itself whenever the Arduino gets power. To change settings (for example to add phone numbers):

1. Edit `firmware/weather_station/config.h`.
2. Stop the dashboard, because uploading also needs the USB connection.
3. Upload it:
    - **Windows:** run `run.bat --upload`. It downloads everything it needs the first time, loads the program, then starts the dashboard again.
    - **Any computer:** use the Arduino IDE with these libraries: **U8g2**, **DHT sensor library** and **Adafruit Unified Sensor**.

The module tests

Open each sketch in `tests/` with the Arduino IDE, upload it, and watch the Serial Monitor at **9600 baud**. `tests/README.md` lists what a pass looks like for each test.

---

7. The GSM module problem

The first SIM800L GSM module turned out to be **faulty**. It switches on (its light blinks normally) but never answers any command. We checked everything else step by step:

#	What we checked	Result
1	Power supply at the module	✓ 4.0 V, steady
2	Common ground	✓ 0 V difference
3	Arduino communication pins (D7/D8)	✓ Working (loop-back test)
4	Level shifter	✓ Working (loop-back test)
5	Every communication speed (1200–115200 baud)	× No response at any speed
6	Transmit/receive wires both ways round	× No response either way
7	Bypassing the level shifter with resistors	× No response
8	Solder joints (checked, then resoldered)	× No change
9	Forced restart of the module	× No change
10	**Voltage on the module's output pin**	**0 V instead of about 2.8 V**

Conclusion: the module's communication output is dead, so it can never reply. Our wiring, power and Arduino are fine. **A replacement module is needed.**

When the new module arrives:

1. Check the LM2596 is still at 4.0 V before connecting it.
2. Check its TXD pin reads about 2.8 V.
3. Run test 07, which works without a SIM card.
4. Test the SIM in a phone first: it must be active, have SMS credit, and have its PIN lock switched off.
5. Switch off, insert the SIM, then run tests 07, 08 and 09.
6. Put the real phone numbers in `config.h` and upload the station program.

Backup plan: SMS through an Android phone

In case the replacement module doesn't arrive in time, the dashboard has a **backup SMS service** that uses a spare **Android phone** instead of the GSM module:

```
Approved phone ──SMS──► Android phone ◄── laptop checks for new texts every 15 s
   (WEATHER)            (gateway app)  ──► laptop builds the reply from the
                                           station's readings and sends it
                                           back through the phone
```

- The phone runs a free app, **SMS Gateway for Android**, and is on the **same Wi-Fi** as the laptop.
- Texting `WEATHER`, `STATUS` or `HELP` to the **phone's number** gets the same kind of reply as the station would send. Alerts (rain, temperature, storm) also go out through the phone.
- **Limitation:** unlike the GSM module, this needs the laptop running, the Arduino connected to it, and Wi-Fi. That's why it's a backup and not the main design.
- It's switched off until it's set up. You enter the phone's address, username, password and the approved numbers in the dashboard's **SMS phone** card and press Save. They can be changed there at any time. Full steps are in `dashboard/README.md` ("Backup SMS through an Android phone"). When it's on, the dashboard header shows **SMS phone: ok**, and the SMS phone card shows whether the phone is connected.

---

8. Things still to do

☐ Get and test a replacement SIM800L GSM module (section 7).

☐ Test sending and receiving SMS, alerts and the forecast in replies.

☑ Set up the Android phone backup (section 7): the dashboard connects to the phone.

☐ Test the phone backup with real texts: `WEATHER`, `STATUS` and `HELP` from an approved number, and one from a number that isn't approved (no reply expected).

☐ Find out why the rain sensor reads about 630 instead of about 1010 when dry (wet plate, loose wire or dirty tracks).

☐ Calibrate the rain sensor to three levels: **dry**, **light rain** and **heavy rain**. Record raw values for each, then update the thresholds in `config.h` and tests 06 and 10.

☐ Optional: calibrate the light sensor (dark and bright values) and the DHT11 (against a reference thermometer).

☐ Mount the station outdoors before trusting the comparison with online data. Indoors, humidity differs from outdoor values.

☐ Let the dashboard run for a few days to collect forecast-accuracy results for the report.

---

9. Files in the project

File / folder	What it is
`README.md`	Short technical overview
`PROJECT_GUIDE.md`	This guide
`sms-weather-station-notes.md`	Original design notes and wiring details

File / folder	What it is
`claude-code-brief.md`	Original work plan
`tests/`	The ten module test sketches + their README
`firmware/weather_station/`	The station program (`weather_station.ino`) and its settings (`config.h`)
`dashboard/`	The dashboard (web app, fake data generator, phone SMS backup, self-check, README)
`run.sh` / `run.bat`	One-command start for Linux/macOS and Windows
`report/`	The technical project report (`PROJECT_REPORT.md` / `.pdf`), its figures, the reference list and the script that builds the PDF

---

10. Glossary

Term	Meaning
Arduino	A small, cheap programmable computer board for electronics projects
Sketch	A program for an Arduino
GSM / 2G	The mobile-phone network standard the SIM800L uses for calls and SMS
AT command	A short text command a GSM module understands, e.g. `AT` ("are you there?"), answered with `OK`
Baud rate	The speed of a serial connection. Both sides must use the same one (here 9600).
Serial / USB serial	The text connection between the Arduino and the laptop, or between the Arduino and the GSM module
TXD / RXD	A device's "transmit" and "receive" pins. One side's TXD connects to the other side's RXD.
Level shifter	Converts signals between 5 V (Arduino) and lower-voltage (GSM module) electronics
Common ground	Joining the ground (minus) of all power supplies so signals have the same reference
I2C	A two-wire connection (SDA, SCL) used by the OLED screen. Each device has an address, e.g. 0x3C.
JSON	A simple text format for data, e.g. `{"t":24.5,"h":61}`. The station uses it to send readings to the laptop.
Open-Meteo	A free online weather service; no account or key needed

Term	Meaning
MAE (mean absolute error)	The average size of a forecast's error, ignoring whether it was too high or too low
Dashboard	The web page on the laptop showing all the data
SQLite / database	The file (`data.db`) where the dashboard stores all readings and forecasts
RAM	The Arduino's working memory, only 2 KB on the Uno